Fenêtres graphiques avec Tkinter

Introduction

- Le module *Tkinter* est inclu dans les distributions Python (pas besoin de faire d'installation de module externe)
- *Tkinter* contrôle la bibliothèque graphique Tk (*Tool Kit*), *Tkinter* signifiant *tk interface*.
- Cette bibliothèque Tk peut être également contrôlée par d'autres langages (Tcl, perl, etc.).
- Il existe beaucoup d'autres modules pour construire des applications graphiques. Par exemple : wxpython, PyQt, PyGObject, etc

Programmation événementielle

- Le gestionnaire d'événements est un genre de « boucle infinie » qui surveille la moindre action de la part de l'utilisateur. C'est lui qui déclenchera une action lors de l'interaction de l'utilisateur avec des éléments de l'interface graphique (GUI).
- Ce mécanisme de gestion par événements fera appel à des *callback*.
- Une fonction *callback* est une fonction passée en argument d'une autre fonction.

Python 3.6
(known limitations)

```
1  #callback.py
2  def fct_callback(arg):
3      print(f"J'aime bien le {arg} !")
4
5
6  def une_fct(ma_callback):
7      print("Je m'apprête à appeler la fonction callback :")
8      ma_callback("psyllium")
9      print("Je termine.")
10
11 if __name__ == "__main__":
12     une_fct(fct_callback)
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Print output (drag lower right corner to resize)

```
Je m'apprête à appeler la fonction callback :
```

Frames | Objects

Global frame
- fct_callback → function fct_callback(arg)
- une_fct → function une_fct(ma_callback)

une_fct
- ma_callback → function fct_callback(arg)

fct_callback
- arg: "psyllium"

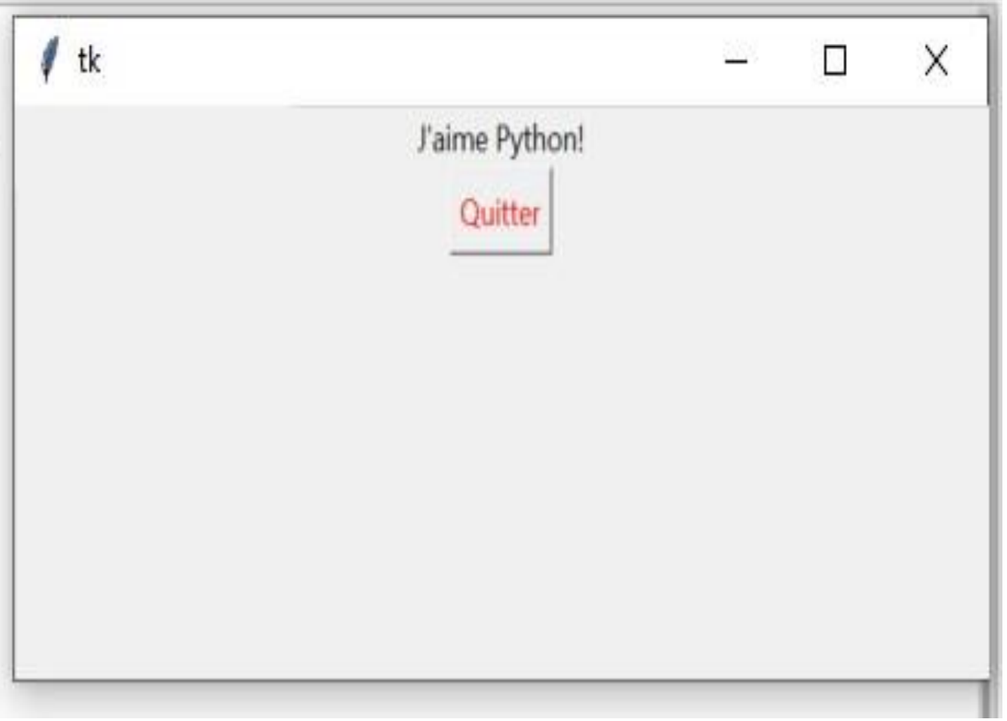
Example : callback.py

La bibliothèque Tkinter

Example de base dans l’interpréteur:

Console

```
>>> import tkinter as tk
>>> racine = tk.Tk() #objet principal Tk
>>> etiquette = tk.Label(racine, text="J'aime Python !") #objet Label
>>> bouton = tk.Button(racine, text="Quitter", fg="red",command=racine.destroy) #objet Button
>>> etiquette.pack() #on rattache le Label à la fenêtre principale
>>> bouton.pack() #on rattache le Button à la fenêtre principale
>>>
```



- Une fois l'appel à la boucle principale *racine.mainloop(),* la fenêtre apparaît
- Notez le callback à la function *destroy()* dans les arguments du bouton

Tkinter dans un script

```python
#baseTkinter.py
import tkinter as tk

racine = tk.Tk()
etiquette = tk.Label(racine, text="J'aime Python !")
bouton = tk.Button(racine, text="Quitter", command=racine.quit) #quitte la mainloop, ne détruit pas
bouton["fg"] = "red" #autre façon de modifier les paramètres d'un widget, a postiori
etiquette.pack()
bouton.pack()
racine.mainloop() #gestion événementielle
print("C'est terminé !") #ne s'affichera que lorsqu'on quittera la boucle principale
```

Example : baseTkinter.py

Tkinter avec une classe

```python
#classeTkinter.py
import tkinter as tk

class Application(tk.Tk):
    def __init__(self):
        tk.Tk.__init__(self)
        self.creer_widgets() #on crée et rattache les widgets en même temps que l'application

    def creer_widgets(self):
        self.etiquette = tk.Label(self, text="J'aime Python !")
        self.bouton = tk.Button(self, text="Quitter", fg = "red", command=self.quit) # callback à quit()
        self.etiquette.pack()
        self.bouton.pack()


if __name__ == "__main__":
    app = Application()
    app.title("Ma Première Application Tkinter")
    app.mainloop()
```

Example : classeTkinter.py

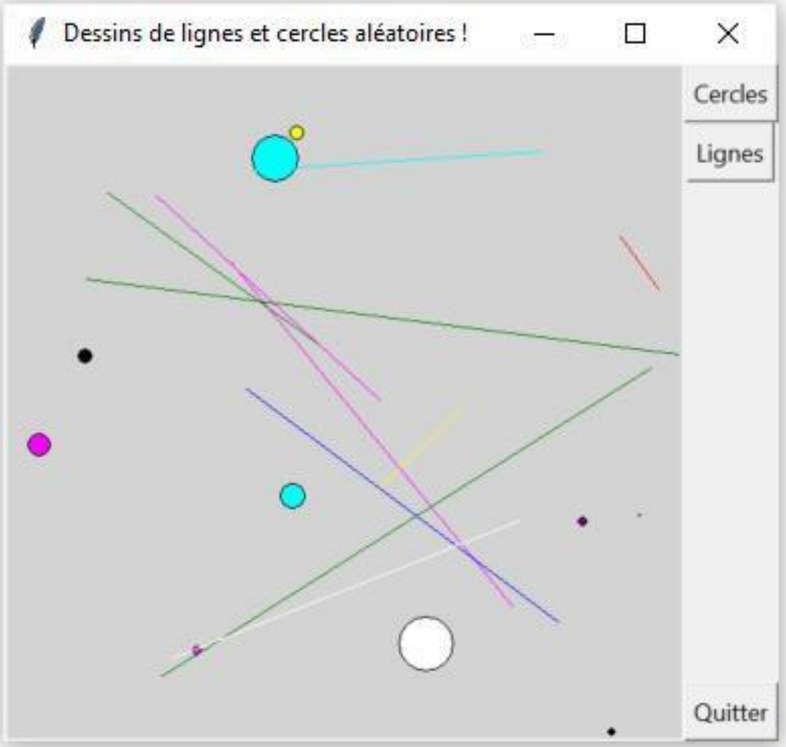
Dessiner avec Tkinter: la classe Canvas (1)

```python
#dessinTkinter.py
import tkinter as tk

racine = tk.Tk()
canv = tk.Canvas(racine, bg="white", height=200, width=200) #Canvas ( master, option=value, ... )
canv.pack()
canv.create_oval(10, 10, 190, 190, outline="red", width=4) #canvas.create_oval(x0, y0, x1, y1, options)
canv.create_line(0, 0, 200, 200, fill="black", width=10) #canvas.create_line(x0, y0, x1, y1, ..., xn, yn, options)
canv.create_line(0, 200, 200, 0, fill="black", width=10)
racine.mainloop()
```

Example : dessinTkinter.py

Dessiner avec Tkinter: la classe Canvas (2)

```python
#dessinClasse.py
import tkinter as tk
import random as rd # pour générer des coordonnées et couleurs aléatoires

class AppliCanevas(tk.Tk):
    taille = 500
    couleurs = ["black", "red", "green", "blue", "yellow", "magenta","cyan", "white", "purple"]
    diametreMax = 50
    maxDessins = 10
    def __init__(self):
        tk.Tk.__init__(self)
        self.size = AppliCanevas.taille # taille de mon application
        self.creer_widgets()

    def creer_widgets(self):
        # création du canevas
        self.canv = tk.Canvas(self, bg="light gray", height=self.size, width=self.size)
        self.canv.pack(side=tk.LEFT) #TOP (défaut), BOTTOM, ou RIGHT
        # boutons
        self.bouton_cercles = tk.Button(self, text="Cercles", command=self.dessine_cercles) #callback
        self.bouton_cercles.pack(side=tk.TOP)
        self.bouton_lignes = tk.Button(self, text="Lignes", command=self.dessine_lignes) #callback
        self.bouton_lignes.pack()
        self.bouton_quitter = tk.Button(self, text="Quitter",command=self.quit)#callback
        self.bouton_quitter.pack(side=tk.BOTTOM)

    def couleur_aleatoire(self):
        return rd.choice(AppliCanevas.couleurs)

    def dessine_cercles(self):
        for i in range(AppliCanevas.maxDessins):
            # génère une coordonnée (deux nombres) aléatoire
            x, y = [rd.randint(1, self.size) for j in range(2)]
            diametre = rd.randint(1, AppliCanevas.diametreMax)
            self.canv.create_oval(x, y, x+diametre, y+diametre,fill=self.couleur_aleatoire())

    def dessine_lignes(self):
        for i in range(AppliCanevas.maxDessins):
            # génère une liste de deux coordonnées (quatre nombres) aléatoires
            x, y, x2, y2 = [rd.randint(1, self.size) for j in range(4)]
            self.canv.create_line(x, y, x2, y2, fill=self.couleur_aleatoire())

if __name__ == "__main__":
    app = AppliCanevas()
    app.title("Dessins de lignes et cercles aléatoires !")
    app.mainloop()
```



Dessiner avec Tkinter: la classe Canvas (3.1)

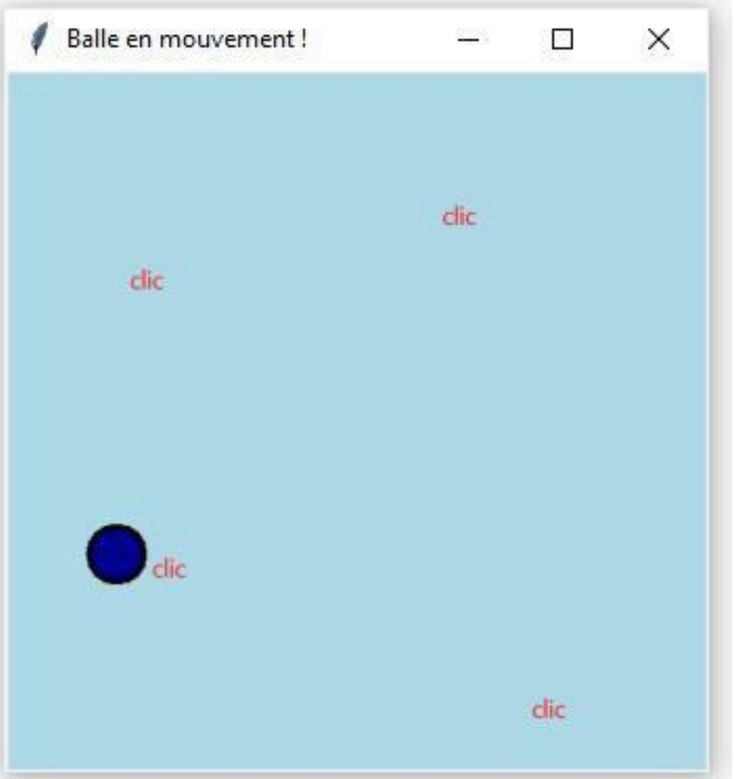
```python
# balleMouvementTkinter.py
#  clic gauche: agrandir la balle
#  clic droit: rapetisser la balle
#  clic central: relancer la balle aléatoirement à ce point
#  touche Tab: quitte l'application baballe

import tkinter as tk
import random as rd
import time

class AppliBalle(tk.Tk):
    xDepart = 250
    yDepart = 250
    rayonBalle = 50
    pasDeplacement = 20
    tailleCanevas = 500
    couleurCanevas = 'light blue'
    couleurBalle = 'dark blue'
    periodeDeplacement = 25
    deplacementRedemarrage = [-30, -20, -10, 10, 20, 30]
    tailleBalleMinimale = tailleBalleChangement = 10
    tailleBalleMaximalle = 200
    def __init__(self):
        # Constructeur de l'application
        tk.Tk.__init__(self)
        # Coordonnées de la balle au départ
        self.x, self.y = AppliBalle.xDepart, AppliBalle.yDepart
        # Rayon de la balle.
        self.size = AppliBalle.rayonBalle
        # Envergure d'un déplacement de la balle.
        self.dx, self.dy = AppliBalle.pasDeplacement, AppliBalle.pasDeplacement
        # Création et packing du canevas.
        self.canv = tk.Canvas(self, bg=AppliBalle.couleurCanevas, height=AppliBalle.tailleCanevas, width=AppliBalle.tailleCanevas)
        self.canv.pack()
        # Création de la balle.
        self.balle = self.canv.create_oval(self.x, self.y, self.x+self.size, self.y+self.size, width=4, fill=AppliBalle.couleurBalle)
        # Binding des actions.
        self.canv.bind("<Button-1>", self.incrementer)
        self.canv.bind("<Button-2>", self.redemarrer)
        self.canv.bind("<Button-3>", self.decrementer)
        self.bind("<Tab>", self.arreter)
        # Lancer la balle.
        self.deplacer()

```

Dessiner avec Tkinter: la classe Canvas (3.2)

```
44
45      def deplacer(self):
46          # Incrémenter les coordonées de la balle.
47          self.x += self.dx
48          self.y += self.dy
49          # Vérifier que la balle ne sort pas du canevas.
50          if self.x < AppliBalle.rayonBalle/2:
51              self.dx = abs(self.dx)
52          if self.x > AppliBalle.tailleCanevas-self.size-AppliBalle.rayonBalle/2:
53              self.dx = -abs(self.dx)
54          if self.y < AppliBalle.rayonBalle/2:
55              self.dy = abs(self.dy)
56          if self.y > AppliBalle.tailleCanevas-self.size-AppliBalle.rayonBalle/2:
57              self.dy = -abs(self.dy)
58          # Mise à jour des coordonnées.
59          self.canv.coords(self.balle, self.x, self.y, self.x+self.size, self.y+self.size)
60          # Rappel de deplacer périodiquement. C'est la même chose que:
61          # import time
62          # ...
63          # while True:
64          #   move()
65          #   time.sleep(AppliBalle.periodeDeplacement/1000) # attendre AppliBalle.periodeDeplacement ms
66          self.after(AppliBalle.periodeDeplacement, self.deplacer)
```

Dessiner avec Tkinter: la classe Canvas (3.3)

```python
    def redemarrer(self, clic):
        #Relance la balle dans une direction aléatoire au point du clic.
        self.x = clic.x
        self.y = clic.y
        self.canv.create_text(self.x, self.y, text="clic", fill="red")
        self.dx = rd.choice(AppliBalle.deplacementRedemarrage)
        self.dy = rd.choice(AppliBalle.deplacementRedemarrage)

    def incrementer(self, lclic):
        #Augmente la taille de la balle.
        self.size += AppliBalle.tailleBalleChangement
        if self.size > AppliBalle.tailleBalleMaximale:
            self.size = AppliBalle.tailleBalleMaximale

    def decrementer(self, rclick):
        #Diminue la taille de la balle.
        self.size -= AppliBalle.tailleBalleChangement
        if self.size < AppliBalle.tailleBalleMinimale:
            self.size = AppliBalle.tailleBalleMinimale

    def arreter(self, esc):
        #Quitte l'application.
        self.quit()

if __name__ == "__main__":
    myapp = AppliBalle()
    myapp.title("Balle en mouvement !")
    print("Avant mainloop")
    myapp.mainloop()
    print("Après mainloop")
```



Example : balleMouvementTkinter.py

Afficher une image avec Tkinter

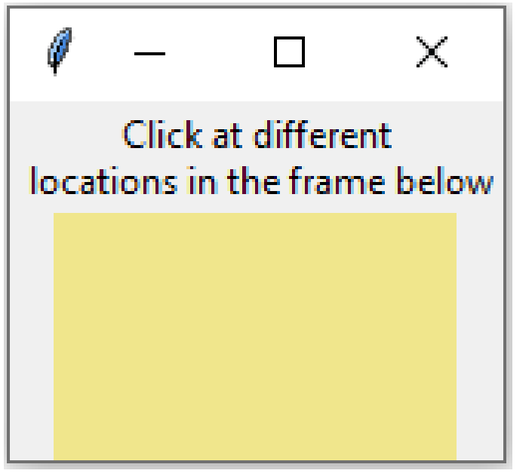
Installez le module pillow : pip3 install pillow

```python
from tkinter import *
import tkinter
import tkinter.messagebox
from PIL import Image
from PIL import ImageTk

master = Tk()

def callback():
    print("click!")

width = 400
height = 400
img = Image.open("alert.png")
img = img.resize((width,height), Image.ANTIALIAS)
photoImg =  ImageTk.PhotoImage(img)
b = Button(master,image=photoImg, command=callback)
b.pack()
mainloop()
```

Example : display_image.py

Position d’un click avec Tkinter

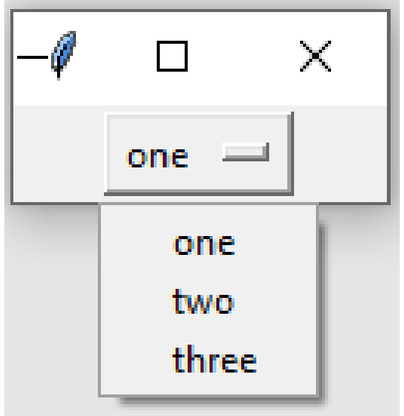
```python
import tkinter as tk
root = tk.Tk()
tk.Label(root, text='Click at different\n locations in the frame below').pack()

def callback(event):
    print(dir(event))
    print("you clicked at", event.x, event.y)

frame = tk.Frame(root, bg='khaki', width=130, height=80)
frame.bind("<Button-1>", callback)
frame.pack()
root.mainloop()
```



Example : display_image.py

Liste déroulante avec Tkinter

```python
from tkinter import *

master = Tk()

variable = StringVar(master)
variable.set("one") # default value

w = OptionMenu(master, variable, "one", "two", "three")
w.pack()

mainloop()
```



Example : dropdown.py

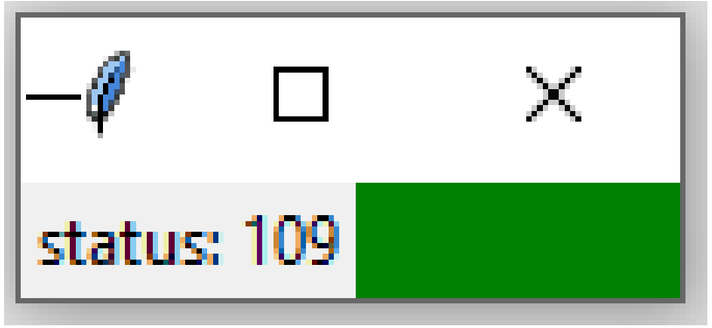
Liste déroulante avec bouton avec Tkinter

```python
from tkinter import *

master = Tk()

var = StringVar(master)
var.set("one") # initial value

option = OptionMenu(master, var, "one", "two", "three", "four")
option.pack()

def ok():
    print("value is: "+var.get())
    master.quit()

button = Button(master, text="OK", command=ok)
button.pack()

mainloop()
```

Example : droplist_ok.py

Étiquette avec Tkinter

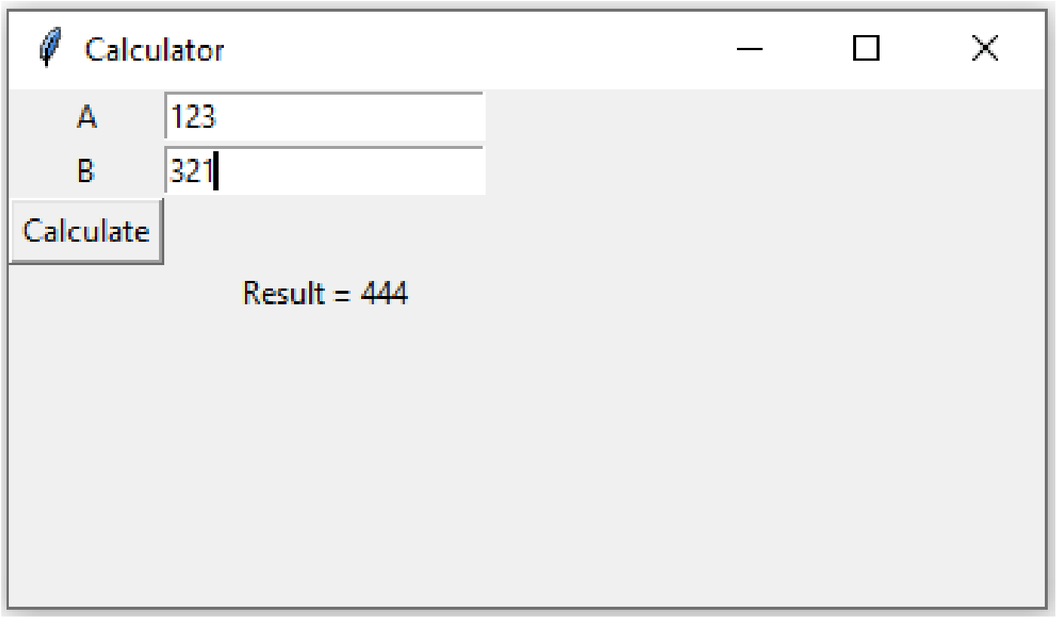
```python
import tkinter as tk
import threading
import time

class SampleApp(tk.Tk):
    def __init__(self, *args, **kwargs):
        # initialize the main window
        tk.Tk.__init__(self, *args, **kwargs)

        # add a container which will take all the widgets
        container = tk.Frame(self, bg="green")
        container.pack(side="top", fill="both", expand=True)

        # Add a titlebar object (defined below)
        self.titleBar = TitleBar(container, controller=self)
        self.titleBar.grid(row=0, column=0, columnspan=2, sticky=tk.N+tk.W+tk.E)

class TitleBar(tk.Frame):
    def __init__(self, parent, controller):
        tk.Frame.__init__(self, parent)
        self.controller = controller

        # the title bar contains only one label element
        self.titleLabel = tk.Label(self, text="This is the initial text")
        self.titleLabel.pack(side=tk.LEFT)

# Define a thread that runs in the background to perform intensive calculations
class MyTestThread(threading.Thread):
    def run(self):
        for i in range(10):
            time.sleep(1)
            a = i+100        # intensive calculation

            # from time to time: inform use about status
            print(a)      # printing to console works fine
            app.titleBar.titleLabel['text'] = "status: " + str(a)

if __name__ == "__main__":
    app = SampleApp()

    #app.titleBar.titleLabel['text'] = "test 2"
    t = MyTestThread()
    t.start()

    app.mainloop()
```



Example : label_increment.py

Entrée avec Tkinter

```python
import tkinter as tk
from functools import partial

def call_result(label_result, n1, n2):
    num1 = (n1.get())
    num2 = (n2.get())
    result = int(num1)+int(num2)
    label_result.config(text="Result = %d" % result)
    return

root = tk.Tk()
root.geometry('400x200+100+200')

root.title('Calculator')

number1 = tk.StringVar()
number2 = tk.StringVar()

labelNum1 = tk.Label(root, text="A").grid(row=1, column=0)

labelNum2 = tk.Label(root, text="B").grid(row=2, column=0)

labelResult = tk.Label(root)

labelResult.grid(row=7, column=2)

entryNum1 = tk.Entry(root, textvariable=number1).grid(row=1, column=2)

entryNum2 = tk.Entry(root, textvariable=number2).grid(row=2, column=2)

call_result = partial(call_result, labelResult, number1, number2)

buttonCal = tk.Button(root, text="Calculate", command=call_result).grid(row=3, column=0)

root.mainloop()
```



Example : entry_sum.py

D’autres widgets

- **Checkbutton** : affiche des cases à cocher.
- **Entry** : demande à l'utilisateur de saisir une valeur / une phrase.
- **Listbox** : affiche une liste d'options à choisir .
- **Radiobutton** : implémente des « boutons radio ».
- **Menubutton** et **Menu** : affiche des menus déroulants.
- **Message** : affiche un message sur plusieurs lignes (extensions du widget Label).
- **Scale** : affiche une règle graduée pour que l'utilisateur choisisse parmi une échelle de valeurs.
- **Scrollbar** : affiche des ascenseurs (horizontaux et verticaux).
- **Text** : crée une zone de texte dans lequel l'utilisateur peut saisir un texte sur plusieurs lignes.
- **Spinbox** : sélectionne une valeur parmi une liste de valeurs.
- **tkMessageBox** : affiche une boîte avec un message

Il existe également une extension de Tkinter nommée **ttk**, réimplémentant la plupart des widgets de base de Tkinter et qui en propose de nouveaux (**Combobox, Notebook**, **Progressbar, Separator**, **Sizegrip** et **Treeview**). Typiquement, si vous utilisez ttk, nous vous conseillons d'utiliser les widgets ttk en priorité, et pour ceux qui n'existent pas dans ttk, ceux de Tkinter (comme Canvas qui n'existe que dans Tkinter). Vous pouvez importer le sous-module ttk de cette manière : **import tkinter.ttk as ttk**

Références supplémentaires

- Tkinter-Tutorial
- Tkinter-By-Example
- Vidéos